

M03 Canonical Lesson Book

16 lessons - video first, then close the video and use this book as the complete solo teacher. If any external video becomes unavailable, the explanation, demo, guided task, recovery and independent proof below remain sufficient.

DE03-01 - INNER JOIN and LEFT JOIN

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 51:10-1:08:04

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Join related tables while choosing INNER vs LEFT from the business rule and stating the result grain before trusting the output.

2. Explain - mental model

A JOIN combines rows through a relationship. INNER JOIN keeps matches only. LEFT JOIN preserves every left-side row and supplies NULLs when the right side has no match. The join type is a business eligibility decision, not a style preference.

3. Demonstrate

```
SELECT o.order_id, o.customer_id, c.customer_name, c.tier
FROM sales_orders o
JOIN customers c ON c.customer_id=o.customer_id;
```

```
SELECT o.order_id, r.return_id
FROM sales_orders o
LEFT JOIN returns r ON r.order_id=o.order_id;
```

4. Guided rebuild

Run both joins. Before each one, say the left input grain, right input grain and intended output grain. Count unmatched return rows in the LEFT JOIN.

5. Expected check

Customer join returns 30 order rows because customers is unique by customer_id. LEFT JOIN returns all 30 orders; exactly 5 orders have a return_id in this lab.

6. Recover when it goes wrong

If rows disappear unexpectedly, check whether INNER JOIN was used where the business rule needed unmatched rows preserved. If rows multiply, inspect right-side key uniqueness before adding DISTINCT.

7. Fresh independent transfer

List Completed West orders with order_id, customer_name, tier and net_sales. Then LEFT JOIN returns and show all Completed orders with optional return_id.

8. Validate

No non-West/non-Completed rows in the first query. Second query preserves the Completed order count; returned orders have return_id and others are NULL.

9. Explain back

Explain INNER vs LEFT using one business example and state what unmatched rows mean.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-02 - Cardinality, row multiplication and join validation

ENGINE / TOOL: MySQL 8+

Video/class route: Luke Barousse - Data Engineer Bootcamp (2026)

Segment: 4:16:18 - Data Modeling & JOINS

https://www.youtube.com/watch?v=ol9_NnC9-cc

RC8 CANONICAL ROUTE; BOOK/LAB FULL FALLBACK

1. Outcome

Predict one-to-one/one-to-many join behavior and detect row multiplication before it corrupts counts or money.

2. Explain - mental model

Cardinality describes how many rows on one side can match the other. A syntactically valid join can still be logically wrong. Validate row count, key uniqueness and at least one numeric total before and after risky joins.

3. Demonstrate

```
SELECT COUNT(*) FROM sales_orders; -- 30
SELECT COUNT(*)
FROM sales_orders o
LEFT JOIN customer_contacts cc ON cc.customer_id=o.customer_id; -- 35
```

4. Guided rebuild

Join sales_orders to customer_contacts by customer_id. Find order_ids that appear more than once and connect the extra rows to customers having multiple contact types.

5. Expected check

Order rows = 30 before; 35 after the raw contact join. The five extra rows are expected fan-out from multi-contact customers, not five new orders.

6. Recover when it goes wrong

Do not repair fan-out with DISTINCT until you know the desired output grain. Pre-aggregate or choose one right-side row when the business question requires one row per order.

7. Fresh independent transfer

Build a one-row-per-customer contact_count CTE/subquery, join it to orders, and prove the order row count stays 30.

8. Validate

Output key order_id remains unique; row count is 30; contact_count can exceed 1 without duplicating orders.

9. Explain back

Explain why a valid foreign-key join can still inflate metrics.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-03 - UNION and UNION ALL

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 1:08:04-1:15:10

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Stack compatible result sets vertically and choose UNION vs UNION ALL intentionally.

2. Explain - mental model

UNION ALL appends compatible result rows and keeps duplicates. UNION additionally removes duplicate result rows. UNION adds rows; JOIN adds columns. Both branches must align in column count and compatible types.

3. Demonstrate

```
SELECT order_id AS event_id, customer_id, order_date AS event_date, 'Order' AS event_type
FROM sales_orders WHERE status='Completed'
UNION ALL
SELECT ticket_id, customer_id, DATE(created_at), 'Ticket'
FROM support_tickets;
```

4. Guided rebuild

Build the combined order/ticket activity stream. Add ORDER BY event_date,event_id after the UNION ALL.

5. Expected check

The stream has 27 Completed-order events plus 10 ticket events = 37 rows because the event_id/type design keeps them distinct.

6. Recover when it goes wrong

If UNION fails, compare branch column counts and types. If a duplicate unexpectedly disappears, check whether UNION was used where UNION ALL was intended.

7. Fresh independent transfer

Create a second two-branch result with intentionally duplicated labels, compare UNION with UNION ALL, and record the row-count difference.

8. Validate

You can explain exactly which duplicate combination UNION removed and why.

9. Explain back

Explain UNION vs UNION ALL vs JOIN in one minute.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-04 - String functions for cleaning and standardization

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 1:15:10-1:26:39

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Use TRIM/UPPER/LOWER/CONCAT and string matching without hiding the original value or overwriting source data.

2. Explain - mental model

String functions can create cleaned/display values in a query. Keep the raw field beside transformed output while validating. A presentation cleanup is not proof that source data is correct.

3. Demonstrate

```
SELECT customer_id, customer_name,  
       UPPER(customer_name) AS name_upper,  
       CONCAT(customer_name, ' - ', tier) AS label  
FROM customers  
WHERE customer_name LIKE '%Retail%';
```

4. Guided rebuild

Run the example and identify every Retail customer. Add a TRIM demo on a literal string with spaces before touching source fields.

5. Expected check

Expected names include Ravi Retail, Om Retail, Ishita Retail, Saanvi Retail and Sara Retail.

6. Recover when it goes wrong

If a cleanup rule changes meaning, keep it as a derived value and investigate rather than updating the source. Do not use string cleanup as a substitute for a data contract.

7. Fresh independent transfer

Create customer_name_clean and a display label while preserving raw customer_name. Filter names containing Stores or Traders.

8. Validate

Raw and transformed names remain visible; the filter returns only names matching the intended text rule.

9. Explain back

Explain safe query-time standardization versus destructive source cleanup.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-05 - Dates, timestamps and time analysis

ENGINE / TOOL: MySQL 8+

Video/class route: Luke Barousse - Data Engineer Bootcamp (2026)

Segment: 10:16:15 - Functions: Date, Text & NULL (date/time portion)

https://www.youtube.com/watch?v=ol9_NnC9-cc

RC8 CANONICAL ROUTE; BOOK/LAB FULL FALLBACK

1. Outcome

Calculate date differences, extract time parts and explain why date, datetime/timestamp and timezone meaning must be deliberate.

2. Explain - mental model

Dates represent calendar dates; timestamps/datetimes represent moments or local date-times depending on the system. DATEDIFF can answer elapsed-day questions in this MySQL lab. Real systems need an explicit timezone contract before comparing event times.

3. Demonstrate

```
SELECT o.order_id, o.order_date, r.return_date,  
DATEDIFF(r.return_date,o.order_date) AS days_to_return  
FROM sales_orders o  
JOIN returns r ON r.order_id=o.order_id  
ORDER BY days_to_return DESC;
```

4. Guided rebuild

Calculate days_to_return for all returned orders and inspect YEAR/MONTH/DAY on order_date.

5. Expected check

Five returned orders are present; every days_to_return is non-negative in the supplied lab.

6. Recover when it goes wrong

If a calculation mixes DATE and DATETIME unexpectedly, inspect types. Never assume a timezone for a timezone-less value unless the source contract says so.

7. Fresh independent transfer

Summarize Completed net_sales by order_date and identify the dates with the highest daily total. Explain what would change if order timestamps arrived from multiple timezones.

8. Validate

Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed.

9. Explain back

Explain date vs timestamp and one timezone failure mode.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-06 - Subqueries and nested logic

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 1:35:16-1:45:58

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Use scalar/list subqueries while keeping each nested level independently testable.

2. Explain - mental model

A subquery is a query used inside another query. Debug the inner level first. Deep nesting is not a badge of sophistication; readable grain and validation matter more.

3. Demonstrate

```
SELECT order_id, qty*unit_price*(1-discount_pct) AS net_sales
FROM sales_orders
WHERE status='Completed'
AND qty*unit_price*(1-discount_pct) > (
  SELECT AVG(qty*unit_price*(1-discount_pct))
  FROM sales_orders WHERE status='Completed'
);
```

4. Guided rebuild

Run the inner average independently, then the full above-average Completed-order query.

5. Expected check

The inner subquery returns one scalar. Outer rows are all Completed and each net_sales is above that scalar.

6. Recover when it goes wrong

If the subquery returns multiple rows where one value is required, restate what the inner query should represent. Use IN/EXISTS when a set/existence question is intended.

7. Fresh independent transfer

Find customers whose Completed net_sales total is above the average customer Completed total. Build/validate the customer-total layer first.

8. Validate

State inner and outer grain separately and reconcile customer totals to overall Completed net_sales.

9. Explain back

Explain why you test a subquery independently before embedding it.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-07 - CTEs and readable multi-step SQL

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 1:58:55-2:09:07

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Break multi-step SQL into named CTEs and state the grain at each stage.

2. Explain - mental model

A CTE gives a query step a name. Its primary value here is readability and inspectable grain, not automatic performance. Each CTE should have a clear purpose and row meaning.

3. Demonstrate

```
WITH order_sales AS (  
  SELECT order_id, customer_id, region, status,  
         qty*unit_price*(1-discount_pct) AS net_sales  
  FROM sales_orders  
) , customer_sales AS (  
  SELECT customer_id, SUM(net_sales) AS customer_net  
  FROM order_sales WHERE status='Completed'  
  GROUP BY customer_id  
)  
SELECT * FROM customer_sales ORDER BY customer_net DESC;
```

4. Guided rebuild

Build order_sales then customer_sales. Run each stage separately during development and record row counts.

5. Expected check

order_sales = one row/order, 30 rows. customer_sales = one row/customer with Completed activity. Its summed customer_net reconciles to Completed net sales.

6. Recover when it goes wrong

If a CTE changes grain unexpectedly, stop and query only that CTE. Do not keep stacking later logic on an unvalidated intermediate result.

7. Fresh independent transfer

Rewrite your DE03-06 customer-above-average solution with named CTEs and add a final validation CTE/result.

8. Validate

Every CTE has a written grain comment and totals reconcile.

9. Explain back

Explain CTE value without claiming it is always faster.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-08 - Window functions and PARTITION BY

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 1:45:58-1:58:55

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Add group context to detail rows with window functions while distinguishing window grain from GROUP BY grain.

2. Explain - mental model

GROUP BY collapses rows to the group grain. A window calculation can preserve detail rows and attach group-level context. PARTITION BY defines independent calculation groups.

3. Demonstrate

```
WITH x AS (  
  SELECT order_id, region, qty*unit_price*(1-discount_pct) AS net_sales  
  FROM sales_orders WHERE status='Completed'  
)  
SELECT order_id, region, net_sales,  
       AVG(net_sales) OVER(PARTITION BY region) AS region_avg_sales  
FROM x;
```

4. Guided rebuild

Add region average net_sales beside every Completed order. Compare row count to a GROUP BY region query.

5. Expected check

Window result preserves 27 Completed detail rows; GROUP BY region produces 4 rows.

6. Recover when it goes wrong

If detail rows disappear, check whether GROUP BY was used instead of a window. If partition totals are wrong, inspect the partition key and eligibility filter.

7. Fresh independent transfer

Add each customer's Completed order count and customer net-sales total beside each of their Completed orders.

8. Validate

Detail row count stays 27; a separate GROUP BY customer query matches the window totals.

9. Explain back

Explain PARTITION BY and why windows preserve detail.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-09 - ROW_NUMBER, ranking and LAG/LEAD

ENGINE / TOOL: MySQL 8+

Video/class route: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours

Segment: 1:45:58-1:58:55

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT WEB CHECK 2026-09-10

1. Outcome

Use deterministic ROW_NUMBER/ranking and LAG/LEAD for latest-row, top-N and previous/next-row analysis.

2. Explain - mental model

ROW_NUMBER assigns a unique sequence based on the ORDER BY you define. RANK/DENSE_RANK treat ties differently. LAG/LEAD reaches a previous/next row within a partition/order. Add deterministic tie-breakers when sequence matters.

3. Demonstrate

```
WITH t AS (  
  SELECT ticket_id, customer_id, created_at, status,  
         ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY created_at DESC, ticket_id DESC) AS rn  
  FROM support_tickets  
)  
SELECT * FROM t WHERE rn=1;
```

4. Guided rebuild

Return latest support ticket per customer. Then add LAG(order_date) to each customer's orders.

5. Expected check

Latest-ticket query returns one row per customer who has tickets. Sequence has an explicit ticket_id tie-breaker.

6. Recover when it goes wrong

If latest-row output is nondeterministic on equal timestamps, add a stable tie-breaker. If LAG crosses customers, add PARTITION BY customer_id.

7. Fresh independent transfer

Rank Completed orders by net_sales within region and return top 2 per region with deterministic order_id tie-breaker.

8. Validate

At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately.

9. Explain back

Explain ROW_NUMBER vs RANK and one LAG use case.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-10 - Advanced aggregation and multi-grain reasoning

ENGINE / TOOL: MySQL 8+

Video route: Luke Barousse - SQL for Data Analytics - Intermediate Course + Project

Segment: 37:44-1:04:47 aggregation; 1:04:47-1:25:28 advanced segmentation

<https://www.youtube.com/watch?v=QKIGsShyEsQ>

Verified 2026-09-10; corrected timestamp/URL parity. Book owns multi-grain proof.

1. Outcome

Combine metrics from different source grains without multiplying them and defend the chosen final grain.

2. Explain - mental model

Orders, returns and tickets are not automatically the same grain. Compute each metric at a stable grain first, then join summaries at the intended output grain. COUNT(DISTINCT ...) can sometimes protect a count, but it does not magically repair multiplied monetary sums.

3. Demonstrate

```
WITH order_region AS (  
  SELECT region, COUNT(*) AS orders_count, SUM(qty*unit_price*(1-discount_pct)) AS net_sales  
  FROM sales_orders WHERE status='Completed' GROUP BY region  
) , return_region AS (  
  SELECT o.region, COUNT(*) AS returned_orders  
  FROM returns r JOIN sales_orders o ON o.order_id=r.order_id GROUP BY o.region  
)  
SELECT o.*, COALESCE(r.returned_orders, 0) AS returned_orders  
FROM order_region o LEFT JOIN return_region r USING(region);
```

4. Guided rebuild

Build order and return metrics separately by region, then combine them. Explain why directly joining orders to tickets by customer_id could multiply both sides.

5. Expected check

Final result is exactly one row/region. Sum of regional net_sales reconciles to Completed net_sales. Returned-order counts sum to 5.

6. Recover when it goes wrong

If totals inflate after a join, inspect source/result grain and pre-aggregate the many-side metrics before joining. Do not hide the problem with DISTINCT on the final output.

7. Fresh independent transfer

Create region metrics: Completed orders, Completed net_sales, returned orders, ticket count. Use separate grain-safe CTEs and combine at one row/region.

8. Validate

4 final rows; each component total reconciles independently; document any customer-to-region assumption.

9. Explain back

Explain why multi-grain analysis is an engineering correctness problem, not just SQL style.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-11 - PostgreSQL setup, schemas and dialect awareness

ENGINE / TOOL: PostgreSQL

Video route: freeCodeCamp - PostgreSQL Tutorial for Beginners

Segment: 0:00-17:12 | <https://www.youtube.com/watch?v=SpflwIAYaKk>

Version guidance: PostgreSQL 18.6/current 18 verified 2026-09-10.

PostgreSQL 19 Beta 3 is not the course baseline. Install only now, at DE03-11.

1. Outcome

Install/verify PostgreSQL when this lesson begins, connect with psql or pgAdmin, create/use a schema, and recognize that SQL dialect/tooling differs from MySQL.

2. Explain - mental model

PostgreSQL is a separate database system from MySQL. Common SQL concepts transfer, but setup, functions, types and commands can differ. A schema is a namespace inside a PostgreSQL database. This lesson deliberately starts a small PostgreSQL mini-lab; it does not replace the deeper M07 PostgreSQL modeling module.

3. Demonstrate

```
SELECT version();
CREATE SCHEMA IF NOT EXISTS m03pg;
SET search_path TO m03pg, public;
SELECT current_schema();
```

4. Guided rebuild

Install/verify the current supported PostgreSQL release from the official project when you reach this lesson. Create database m03_de (or use an existing course DB), run M03_POSTGRESQL_MINI_LAB_SETUP.sql, then run SELECT version() and SHOW search_path.

5. Expected check

You can connect successfully, identify the server version you actually observe, and query m03pg.orders. The mini-lab reports 8 customers and 12 orders.

6. Recover when it goes wrong

If psql/pgAdmin cannot connect, stop at connection setup rather than changing SQL. Verify server service, host/port/database/user and exact error. Do not install PostgreSQL earlier in M02.

7. Fresh independent transfer

In a fresh query tab, set search_path and return order_id, customer_id, status from m03pg.orders. Write two syntax/function differences you have actually observed or verified between the MySQL and PostgreSQL labs.

8. Validate

Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented.

9. Explain back

Explain database vs schema and why SQL portability is not perfect.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-12 - Constraints and transactions

ENGINE / TOOL: PostgreSQL

Video A: freeCodeCamp / Amigoscode PostgreSQL - constraints and foreign keys

2:29:24-2:54:45 + 3:16:41-3:25:04 | <https://www.youtube.com/watch?v=qw--VYLpxG4>

Video B: Harvard CS50 SQL Week 5 - transactions

1:02:29-1:15:57 | <https://cs50.harvard.edu/sql/weeks/5/>

Verified 2026-09-10; book/lab controls PostgreSQL-specific mastery.

1. Outcome

Use primary/foreign/check/not-null constraints and prove COMMIT/ROLLBACK behavior without weakening integrity rules.

2. Explain - mental model

Constraints make invalid states harder to store. A transaction groups changes so they can commit together or roll back. When a constraint rejects a row, the correct response is to understand the rule/data - not remove the constraint to silence the error.

3. Demonstrate

```
BEGIN;
UPDATE m03pg.orders SET status='Cancelled' WHERE order_id='P1001';
SELECT order_id,status FROM m03pg.orders WHERE order_id='P1001';
ROLLBACK;
SELECT order_id,status FROM m03pg.orders WHERE order_id='P1001';
```

4. Guided rebuild

Run the rollback demonstration. Then deliberately attempt a duplicate order_id and an invalid status inside a safe transaction and capture the constraint errors.

5. Expected check

P1001 returns to its original status after ROLLBACK. Invalid/duplicate rows do not remain stored.

6. Recover when it goes wrong

If a failed statement aborts the PostgreSQL transaction, ROLLBACK before continuing. Do not drop constraints to make the test pass.

7. Fresh independent transfer

Insert one valid support_case inside BEGIN; inspect it; ROLLBACK; prove row count returns to the baseline. Explain which constraint protects each critical field.

8. Validate

Baseline row count restored after rollback; no invalid row persists.

9. Explain back

Explain transaction, COMMIT, ROLLBACK and one integrity constraint in plain English.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-13 - Indexes and EXPLAIN basics

ENGINE / TOOL: PostgreSQL

Video route: Luke Barousse - SQL for Data Analytics - Intermediate Course + Project

Segment: 5:10:55-5:42:10 EXPLAIN and query-optimization support

<https://www.youtube.com/watch?v=QKIGsShyEsQ>

Verified 2026-09-10; corrected timestamp/URL parity. PostgreSQL lab is mastery proof.

1. Outcome

Create a query-driven index, read basic EXPLAIN evidence, and avoid claiming speedup that the small learning table cannot prove.

2. Explain - mental model

An index is an additional structure that can help some lookups but costs storage/write maintenance. PostgreSQL chooses plans based on table size/statistics/cost. On tiny course tables, the planner may still prefer a sequential scan. Your job is to report observed plan evidence truthfully.

3. Demonstrate

```
EXPLAIN SELECT * FROM m03pg.orders WHERE customer_id='PC003';
CREATE INDEX IF NOT EXISTS idx_m03_orders_customer_id ON m03pg.orders(customer_id);
ANALYZE m03pg.orders;
EXPLAIN SELECT * FROM m03pg.orders WHERE customer_id='PC003';
```

4. Guided rebuild

Capture EXPLAIN before and after the index. Compare plan nodes/cost/rows estimates you actually see; do not require an Index Scan on a 12-row table.

5. Expected check

Index exists in pg_indexes after creation. Before/after plans are saved. Any claim about planner choice matches observed output.

6. Recover when it goes wrong

If the plan still shows Seq Scan, that is not a failure on a tiny table. Explain table-size/cost reasoning; do not force planner settings just to manufacture an index scan.

7. Fresh independent transfer

Choose one repeated filter/join column from the mini-lab, justify whether an index is sensible, create it only if justified, and save before/after EXPLAIN plus one downside.

8. Validate

Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.”

9. Explain back

Explain what EXPLAIN proves and what it does not prove.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-14 - Views and reusable database logic

ENGINE / TOOL: PostgreSQL

Video route: Luke Barousse - SQL for Data Analytics - Intermediate Course + Project

Segment: 3:45:59-4:06:27 View Intro

<https://www.youtube.com/watch?v=QKIGsShyEsQ>

Verified 2026-09-10; old source had no Views chapter. Book/PostgreSQL lab authoritative.

1. Outcome

Create a view for stable reusable query logic while explaining that a normal view stores a query definition, not an independent copy of the data.

2. Explain - mental model

A normal PostgreSQL view gives a reusable named SELECT. It can simplify consumer queries and centralize logic, but it depends on underlying objects and permissions. A view is not automatically a performance optimization or a materialized dataset.

3. Demonstrate

```
CREATE OR REPLACE VIEW m03pg.v_completed_order_value AS
SELECT o.order_id, o.order_date, o.customer_id, c.customer_name,
SUM(i.qty*i.unit_price) AS gross_sales
FROM m03pg.orders o
JOIN m03pg.customers c USING(customer_id)
JOIN m03pg.order_items i USING(order_id)
WHERE o.status='Completed'
GROUP BY o.order_id, o.order_date, o.customer_id, c.customer_name;
SELECT * FROM m03pg.v_completed_order_value;
```

4. Guided rebuild

Create the completed-order view, query it, then change one underlying row inside a transaction and observe the view before rolling back.

5. Expected check

The view exposes only Completed orders and reflects underlying data because it is a normal view. Rollback returns results to baseline.

6. Recover when it goes wrong

If the view duplicates orders, inspect order_items aggregation and intended view grain. If consumers need a physical cached result, that is a different design topic; do not call a normal view materialized.

7. Fresh independent transfer

Create a view m03pg.v_customer_order_summary at one row/customer with order_count and gross_sales. State dependencies and one limitation.

8. Validate

One row/customer with activity; sum of gross_sales matches an independent aggregate over eligible source rows.

9. Explain back

Explain normal view vs table and why views can improve maintainability without guaranteeing speed.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-15 - Integrated unseen SQL challenge

ENGINE / TOOL: Protected mixed MySQL + PostgreSQL

Assessment route: Independent practice / protected assessment

NO VIDEO BY DESIGN

PRACTICE ONLY - MEDIA CLOSED

1. Outcome

Combine grain/cardinality, CTE/window logic, validation and PostgreSQL database-thinking skills without tutorial support.

2. Explain - mental model

This is an assessment lesson, not another tutorial. A genuine attempt is required before review. You may use the database clients and the supplied source data, but keep videos, solution/review material and AI solving closed.

3. Demonstrate

No worked solution is shown before attempt. Read the challenge brief, state grains/eligibility, build in small validated steps and preserve failed attempts.

4. Guided rebuild

No guided solution. Planning scaffold only: 1) state desired output grain; 2) list source grains; 3) identify fan-out risks; 4) build/validate stages; 5) save result + controls; 6) explain limitations.

5. Expected check

Challenge rubric requires correct grain, no unexplained multiplication, at least one CTE/window pattern, two independent validations, and one PostgreSQL object/transaction/plan evidence item.

6. Recover when it goes wrong

After a saved attempt, open only the matching protected review for weak competencies. Then close review and use the fresh retest variant.

7. Fresh independent transfer

Complete the M03 integrated challenge in 90 minutes. MySQL portion: build a one-row/customer priority table from orders + tickets without fan-out, with latest ticket and customer sales rank. PostgreSQL portion: prove one rollback, one justified index/EXPLAIN comparison and one reusable view.

8. Validate

All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and PostgreSQL evidence is observed/saved.

9. Explain back

Give a 3-minute defense: grain, biggest fan-out risk, validation, Postgres object choice and one limitation.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.

DE03-16 - Timed SQL interview set and explanation

ENGINE / TOOL: Protected mixed SQL

Assessment route: Independent practice / protected assessment

NO VIDEO BY DESIGN

PRACTICE ONLY - MEDIA CLOSED

1. Outcome

Solve a fresh timed SQL set and explain choices clearly under interview-like constraints.

2. Explain - mental model

Timed work tests retrieval and reasoning, not memorization. Correctness includes grain and validation. A query that runs but answers the wrong question is not a pass.

3. Demonstrate

No answer is exposed before attempt. Use a 45-minute timer for the SQL set plus 10-15 minutes for explanation/validation notes.

4. Guided rebuild

No guided solution. Time allocation suggestion: 5 min read/grain, 30 min queries, 10 min validation, then explanation.

5. Expected check

Pass target: $\geq 85/100$, no critical grain/cardinality/unsafe-transaction failure, and clear explanation of at least one window/CTE and one PostgreSQL database object.

6. Recover when it goes wrong

If one domain fails, repair only that domain and take a different timed set. Do not repeat the same questions as mastery evidence.

7. Fresh independent transfer

Timed set: (1) diagnose a fan-out join; (2) top-2 Completed non-returned orders/region; (3) latest ticket/customer; (4) region net-sales + return rate safely; (5) explain transaction rollback and EXPLAIN evidence; (6) defend view vs table.

8. Validate

Save SQL, result controls, time used, confidence, errors and explanation. Review remains closed until the attempt is preserved.

9. Explain back

Answer two random questions from the set aloud without reading your SQL.

10. Mastery evidence

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered.